SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

CLOUD MISCONFIGURATION DETECTION AND AUTOMATED REMEDIATION

A Serverless, Event-Driven Security Control on AWS

Detailed Project Report

Submitted by

Aaron Ninan - 1601012005

Aditya Jagdale - 16010123024

Abdullah Idris Qureshi - 16010123015

B.Tech Computer Engineering, [Year]

K.J. Somaiya School of Engineering, Mumbai

Advanced Cloud Computing Lab Project

October 2026

Platform: Amazon Web Services, Asia Pacific (Mumbai) ap-south-1

Abstract

Cloud data exposures are often traced to configuration mistakes instead of software exploits. A single weakened setting on a storage bucket can leave data exposed until someone notices, and manual review cannot keep up because cloud resources are created and changed within seconds. This project builds a serverless control on Amazon Web Services (AWS) that watches one S3 bucket and repairs a weakened Block Public Access configuration without human intervention.

When the Block Public Access configuration of the bucket is changed, AWS CloudTrail records the API call. An Amazon EventBridge rule on the default event bus matches the event for the monitored bucket and invokes an AWS Lambda function written in Python. The function reads the current configuration and, if any of the four protections (BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, RestrictPublicBuckets) is disabled, enables all four. It then reads the configuration again to verify the fix and writes the result to Amazon CloudWatch Logs. The function runs under an IAM role limited to two S3 actions on one bucket, and the account-level protections stay enabled throughout.

The prototype was tested by invoking the function directly, by sending a synthetic event to the event bus, and in two live runs in which the Block public policy setting was switched off in the console. In both live runs the setting returned to On without manual action. The report documents the design, the implementation, the security analysis, the test results, a troubleshooting record, the limitations and the future work.

Keywords: *cloud security, misconfiguration, AWS, S3 Block Public Access, CloudTrail, EventBridge, Lambda, serverless, automated remediation, least privilege.*

Table of contents

1. Introduction 6
1.1 Background 6
1.2 Problem statement 6
1.3 Objectives 6
1.4 Scope 7
2. Background and related concepts 8
2.1 Cloud misconfiguration and shared responsibility 8
2.2 Amazon S3 Block Public Access 8
2.3 Event-driven detection with CloudTrail and EventBridge 8
2.4 Serverless compute with AWS Lambda 8
2.5 Detective and corrective controls, and verification 9
3. Technology stack 10
3.1 Components 10
3.2 Configuration summary 10
3.3 Why AWS 11
4. System architecture 12
4.1 Architecture overview 12
4.2 Event flow 12
4.3 Event pattern 13
5. Implementation 14
5.1 S3 bucket and Block Public Access 14
5.2 IAM configuration 14
5.3 Lambda function 15
5.4 EventBridge rule and invocation permission 17
5.5 CloudTrail trail 19
5.6 CloudWatch Logs 20
6. Security analysis 21
6.1 Security aspects addressed 21
6.2 Least privilege and limited impact 21
6.3 Threat considerations 21
6.4 Further aspects the same design can cover 22
7. Testing and demonstration 23
7.1 Test strategy 23
7.2 Demonstration procedure 23
7.3 Safety measures during testing 24
8. Results and discussion 25
8.1 Observed results 25
8.2 Troubleshooting record 28
8.3 Discussion 28
8.4 Limitations 29
9. Why cloud security is necessary 30
10. Conclusion and future work 31
10.1 Conclusion 31
10.2 Future work 31
References 32
Appendix A: Lambda function source 33

List of figures

Figure 1: High-level architecture of the remediation system
Figure 2: S3 bucket Block Public Access with all four settings on
Figure 3: Execution role permissions
Figure 4: Decision flow of the remediation function
Figure 5: Lambda function overview
Figure 6: EventBridge rule details and event pattern
Figure 7: EventBridge rule targets
Figure 8: Lambda resource-based policy
Figure 9: CloudTrail trail configuration
Figure 10: Misconfiguration step
Figure 11: CloudTrail event record of the misconfiguration
Figure 12: Matched event in the rule debug log group
Figure 13: Lambda log output
Figure 14: Bucket restored to the secure state
Figure 15: CloudTrail: change followed by remediation
Figure 16: EventBridge rule monitoring

List of tables

Table 1: The four Block Public Access settings
Table 2: Technology components and roles
Table 3: Resources and configuration values
Table 4: Event pattern fields
Table 5: Permissions of the execution role
Table 6: Lambda resource-based policy statement
Table 7: Log groups used
Table 8: Security aspects covered by the demonstration
Table 9: Threat considerations
Table 10: Test cases
Table 11: Measured values
Table 12: Troubleshooting observations
Table 13: Limitations of the prototype

1. Introduction

1.1 Background

Cloud platforms let teams create storage, networks and compute in seconds. The same speed applies to mistakes. A single changed setting on a storage bucket can expose data to the internet, and the person who made the change may not realise it. Industry guidance for cloud security describes a shared responsibility model: the provider secures the underlying infrastructure, and the customer is responsible for how their own resources are configured, who can access them and what data they hold [1].

Amazon S3 offers a safeguard against accidental public exposure called Block Public Access. It consists of four settings that can be applied to a bucket or to a whole account [2]. The safeguard works only while it stays enabled, and anyone with the right permissions can switch it off. Detecting that change quickly, and reversing it, is therefore a security task that does not need to wait for a human reviewer.

Two kinds of control are relevant. A detective control notices that something has gone wrong, and a corrective control puts it right. Event-driven cloud services make it practical to combine the two: an API call generates an event, a rule matches the event, and a small function reacts. This project applies that pattern to one concrete misconfiguration, a weakened S3 Block Public Access configuration.

1.2 Problem statement

Many cloud data exposures come from configuration errors instead of attacks on the software itself. Manual monitoring is slow, and the period during which a bucket is less protected than intended can last hours or days. This project addresses three gaps:

1. A weakened S3 protection can go unnoticed because nobody is watching the configuration continuously.
2. Remediation depends on a person finding the problem, deciding what to do and making the change, which delays the fix.
3. Where remediation is automated, the result must be verified and recorded, otherwise the team cannot show that the fix actually happened.

1.3 Objectives

1. Detect changes to the Block Public Access configuration of a chosen S3 bucket using AWS CloudTrail and Amazon EventBridge.

2. Invoke an AWS Lambda function automatically when such a change is recorded.
3. Restore all four Block Public Access protections whenever any of them is found disabled.
4. Read the configuration again after the change to verify that remediation succeeded.
5. Record the event, the settings found and the result in Amazon CloudWatch Logs.
6. Run the function with the minimum IAM permissions it needs, and keep account-level protections enabled throughout testing.

1.4 Scope

The prototype is deliberately narrow. It protects one bucket in the AWS Asia Pacific (Mumbai) region, ap-south-1, and responds to two API calls: `PutBucketPublicAccessBlock` and `DeletePublicAccessBlock`. It does not inspect bucket policies or ACLs, does not manage encryption or versioning, and does not send notifications. Account-level Block Public Access was left enabled, no public bucket policy or ACL was ever added, and during testing only the Block public policy setting of the bucket was switched off, for short periods. The effort of the project went into the detection and remediation chain and into evidence that it works.

2. Background and related concepts

2.1 Cloud misconfiguration and shared responsibility

A misconfiguration is a resource setting that leaves a system less protected than its owner intends, such as a storage bucket that allows public reads or a firewall rule that opens a port to every address. Cloud providers secure their data centres, hardware and core services, while customers decide how to configure what they deploy [1]. Because configuration is controlled through APIs, it can change at any time and from many places: the console, a command line, an infrastructure template or a script. Continuous detection is therefore more reliable than periodic review.

2.2 Amazon S3 Block Public Access

S3 Block Public Access provides four settings that limit how public access can be granted to buckets and objects [2]. They can be applied to a single bucket or to every bucket in an account. A bucket is fully covered only when all four are enabled, which is why this project treats any single disabled setting as a misconfiguration. The settings are summarised in Table 1.

Table 1: The four Block Public Access settings

Setting	Effect when enabled
BlockPublicAcls	Rejects requests that try to add a public ACL to a bucket or an object
IgnorePublicAcls	Ignores every public ACL on the bucket and on its objects
BlockPublicPolicy	Rejects a bucket policy or access point policy that grants public access
RestrictPublicBuckets	Limits access to a bucket that has a public policy to AWS services and authorised users in the account

2.3 Event-driven detection with CloudTrail and EventBridge

AWS CloudTrail records API activity in an account as events [3]. Calls that change the configuration of a resource, such as the S3 call that sets Block Public Access, are management events. Amazon EventBridge receives events and compares each one with the patterns of its rules. When an event matches, the rule sends it to one or more targets, such as a Lambda function or a CloudWatch log group [4]. A rule on the default event bus therefore acts as a filter that turns a specific API call into a trigger.

2.4 Serverless compute with AWS Lambda

AWS Lambda runs code in response to events without the user managing servers [5]. Two separate permission mechanisms matter here. The function's execution role, assumed by the Lambda service, decides what the code may do to other AWS services. The function's resource-

based policy decides which services or accounts may invoke the function. Both must be correct for an event-driven design to work.

2.5 Detective and corrective controls, and verification

Automatic remediation is only trustworthy if it is safe to repeat and if it checks its own work. The function in this project is idempotent: if the bucket is already secure it changes nothing. It also reads the configuration back after writing it, and raises an error if the read-back does not show all four protections enabled. Least-privilege IAM limits what the function could do even if it contained a defect [6].

3. Technology stack

The solution uses six AWS services, all managed and billed on use, and one programming language. Table 2 lists each component and its role. The detailed configuration values are collected in Table 3.

3.1 Components

Table 2: Technology components and roles

Component	Role in the project
Amazon S3	Holds the protected bucket whose Block Public Access settings are monitored and restored
AWS CloudTrail	Records the S3 API call as a management event; a trail with write management events is configured
Amazon EventBridge	Rule on the default bus matches the event for the monitored bucket and invokes the Lambda function
AWS Lambda	Runs the Python remediation function: check, remediate, verify, log
AWS IAM	Execution role with two S3 actions on one bucket, plus the Lambda resource-based policy for EventBridge
Amazon CloudWatch Logs	Stores the function output and, through a debug target, the raw matched event
Python 3 and boto3	Language and AWS SDK used by the remediation function

3.2 Configuration summary

Table 3: Resources and configuration values

Item	Value
Region	Asia Pacific (Mumbai), ap-south-1
Monitored S3 bucket	acc-demo-bucket-849076101014
Lambda function	S3PublicAccessRemediation (Python, x86_64, timeout 15 seconds)
Lambda runtime version	[Python 3.__ : confirm from the Lambda console]
Execution role	S3MisconfigurationRemediationRole
Custom IAM policy	S3BucketPublicAccessRemediationPolicy
Managed IAM policy	AWSLambdaBasicExecutionRole (CloudWatch Logs only)
EventBridge rule and bus	DetectS3PublicAccessChanges, default event bus
CloudTrail trail	s3-misconfig-trail (multi-region, write management events)
Log groups	/aws/lambda/S3PublicAccessRemediation and /aws/events/s3-rule-debug

3.3 Why AWS

AWS was chosen because the required building blocks exist as managed services that are designed to work together. CloudTrail events flow to EventBridge, EventBridge invokes Lambda, and Lambda logs to CloudWatch with no servers to operate. Billing is based on use, so a demonstration costs very little. IAM allows permissions to be limited to named actions on a named resource, which supports the least-privilege goal of the project.

4. System architecture

4.1 Architecture overview

Figure 1 shows the complete flow. A change to the bucket's Block Public Access configuration is recorded by CloudTrail. EventBridge matches the recorded event and invokes the Lambda function, which reads the configuration, restores it if necessary, verifies the result and logs it. The execution role supplies the function's permissions.

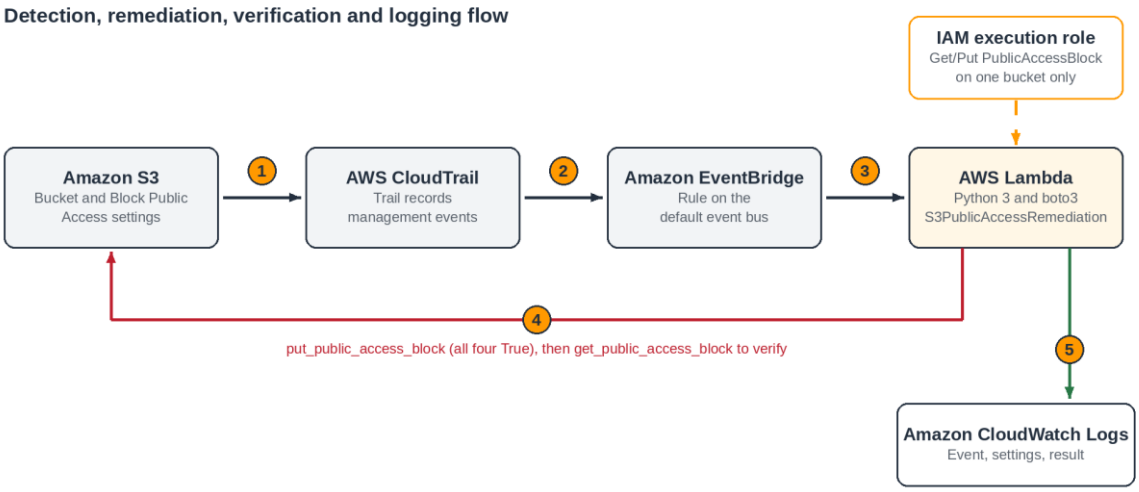


Figure 1: High-level architecture of the remediation system

4.2 Event flow

One misconfiguration passes through the following steps. The numbering corresponds to the numbered markers in Figure 1.

1. A user or script calls `PutBucketPublicAccessBlock` and disables at least one of the four settings.
2. CloudTrail records the call as a management event with the bucket name and the new configuration.
3. EventBridge compares the event with the pattern of the rule `DetectS3PublicAccessChanges`.
4. The rule matches and invokes `S3PublicAccessRemediation`, passing the event as input. A second target writes the same event to a debug log group.
5. The function reads the current configuration. If any setting is not True, it writes all four as True and reads the configuration again to verify.

6. The function logs the event, the settings found and the verified result to CloudWatch Logs. Its own write is also recorded by CloudTrail and triggers one more invocation that finds the bucket already secure and stops.

4.3 Event pattern

The rule matches only the two API calls that change the Block Public Access configuration, and only for the monitored bucket. Including `DeletePublicAccessBlock` covers the case where the whole configuration is removed instead of weakened.

```
{
  "source": ["aws.s3"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["s3.amazonaws.com"],
    "eventName": ["PutBucketPublicAccessBlock",
                  "DeletePublicAccessBlock"],
    "requestParameters": {
      "bucketName": ["acc-demo-bucket-849076101014"]
    }
  }
}
```

Table 4: Event pattern fields

Field	Value	Purpose
source	aws.s3	Events that originate from S3
detail-type	AWS API Call via CloudTrail	Events derived from CloudTrail API records
detail.eventSource	s3.amazonaws.com	Confirms the S3 service API
detail.eventName	PutBucketPublicAccessBlock, DeletePublicAccessBlock	Configuration being changed or removed
detail.requestParameters. bucketName	acc-demo-bucket-849076101014	Restricts the rule to the monitored bucket

5. Implementation

5.1 S3 bucket and Block Public Access

The bucket `acc-demo-bucket-849076101014` was created in ap-south-1 with all four bucket-level protections enabled, and it holds no public bucket policy or ACL. Account-level Block Public Access was left enabled. Figure 2 shows the secure state that the system must always return to.

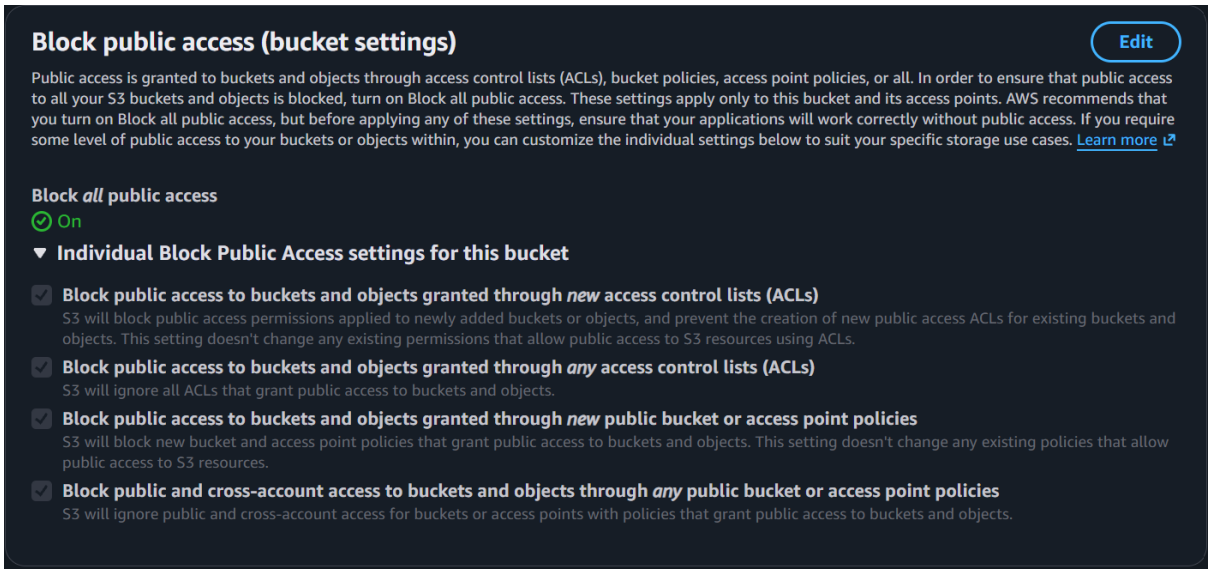


Figure 2: S3 bucket Block Public Access with all four settings on

5.2 IAM configuration

The Lambda function runs under the role `S3MisconfigurationRemediationRole`. Its trust policy lets only the Lambda service assume it.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "lambda.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

Two policies are attached. `AWSLambdaBasicExecutionRole` lets the function write to CloudWatch Logs. The custom policy `S3BucketPublicAccessRemediationPolicy` grants only the two S3 actions the function needs, on the monitored bucket alone (Table 5).

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "s3:GetBucketPublicAccessBlock",
      "s3:PutBucketPublicAccessBlock"
    ],
    "Resource": "arn:aws:s3:::acc-demo-bucket-849076101014"
  }]
}
```

Table 5: Permissions of the execution role

Policy	Allowed actions	Resource
S3BucketPublicAccessRemediationPolicy	s3:GetBucketPublicAccessBlock, s3:PutBucketPublicAccessBlock	The one monitored bucket
AWSLambdaBasicExecutionRole	logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents	CloudWatch Logs
(none)	No s3:*, no lambda:*, no wildcard actions	Not applicable

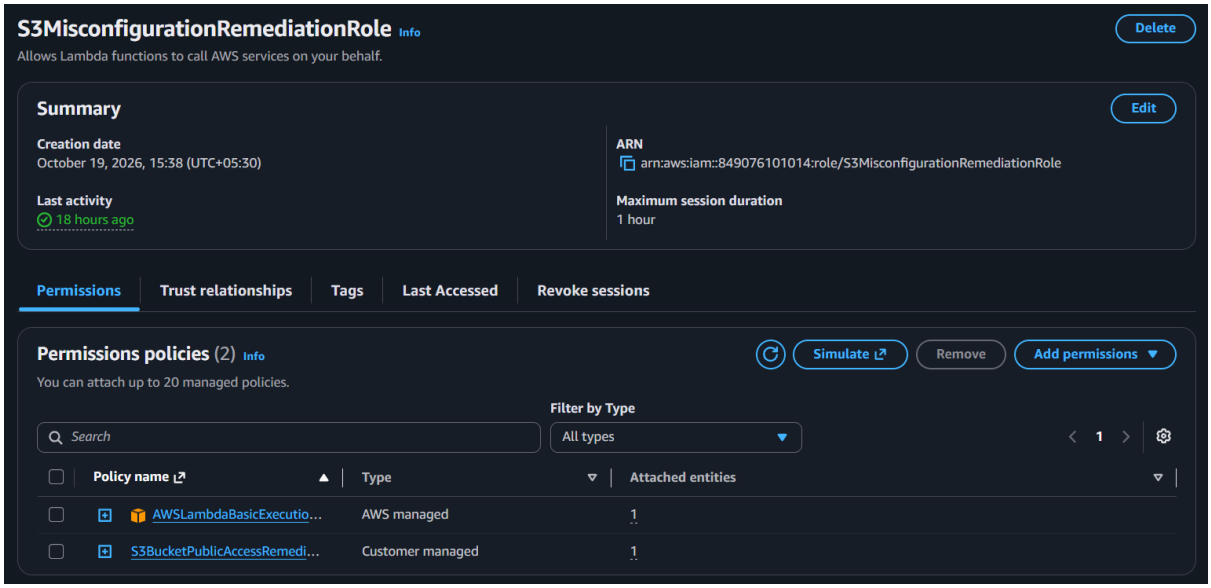


Figure 3: Execution role permissions

5.3 Lambda function

The function is written in Python with the boto3 SDK. Its logic is shown in Figure 4, and the full source is in Appendix A. It first reads `requestParameters.bucketName` from the event and ignores events for other buckets. It then reads the current configuration. A missing configuration, which is what remains after `DeletePublicAccessBlock`, is treated as not

secure. If all four settings are True it exits without changes. Otherwise it writes all four as True, reads them back, and raises an error if verification fails.

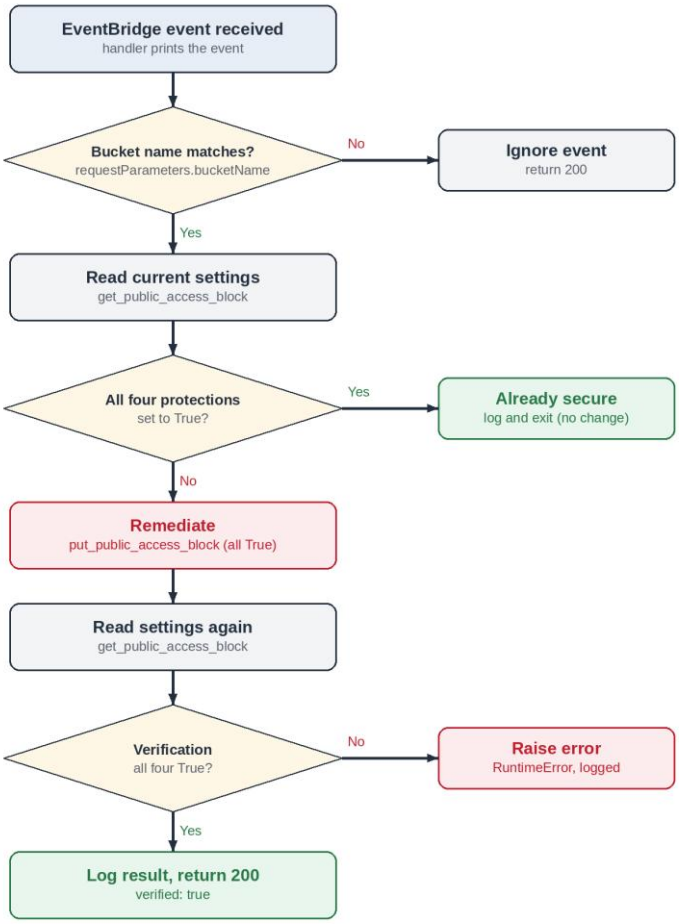


Figure 4: Decision flow of the remediation function

Several design decisions follow from the objectives:

- **Idempotent:** a second invocation on a secure bucket changes nothing, which prevents a remediation loop.
- **Scoped:** the bucket name is fixed in the code, so an event for another bucket is ignored even if the rule were widened.
- **Verified:** the function re-reads the configuration and fails loudly instead of reporting success it has not confirmed.
- **Observable:** the event, the settings found, and the result are printed, so CloudWatch Logs holds the evidence.
- **Short timeout:** a 15-second timeout is enough because the function makes at most four API calls.

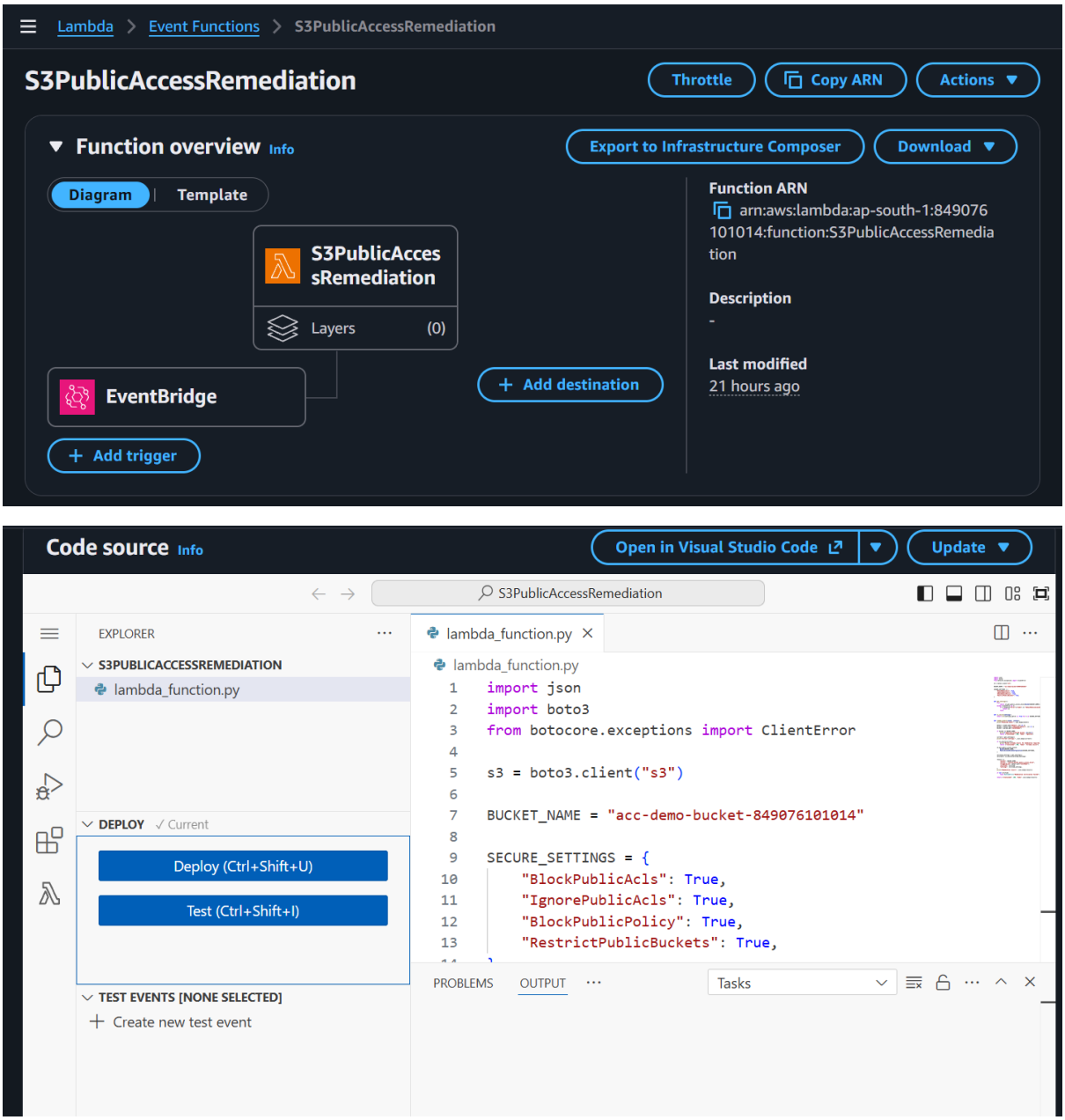


Figure 5: Lambda function overview

5.4 EventBridge rule and invocation permission

The rule `DetectS3PublicAccessChanges` is on the default event bus and uses the pattern in Section 4.3. It has two targets: the Lambda function, and the log group `/aws/events/s3-rule-debug`, which records the exact event EventBridge received and makes matching visible. The console offers to create an execution role for the Lambda target; this was accepted but is not what grants permission to invoke the function.

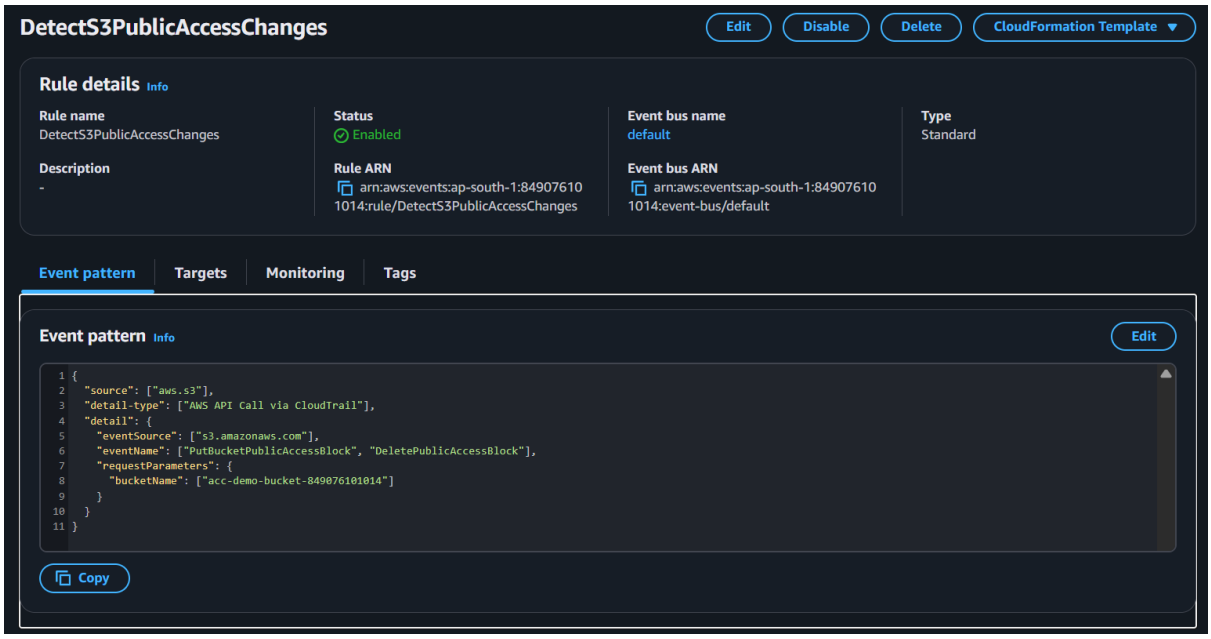


Figure 6: EventBridge rule details and event pattern

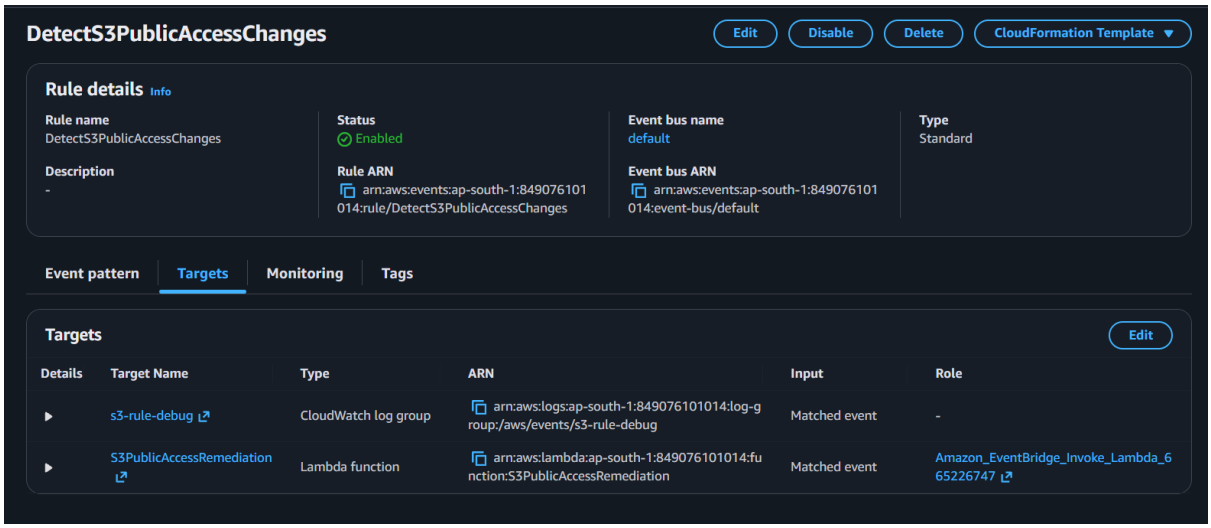


Figure 7: EventBridge rule targets

Permission to invoke the function comes from its resource-based policy (Table 6). Without this statement EventBridge cannot call the function, whatever the rule matches.

Table 6: Lambda resource-based policy statement

Element	Value
Principal	events.amazonaws.com
Action	lambda:InvokeFunction
Resource	arn:aws:lambda:ap-south-1:849076101014:function:S3PublicAccessRemediation

Element	Value
Condition (source ARN)	arn:aws:events:ap-south-1:849076101014:rule/DetectS3PublicAccessChanges

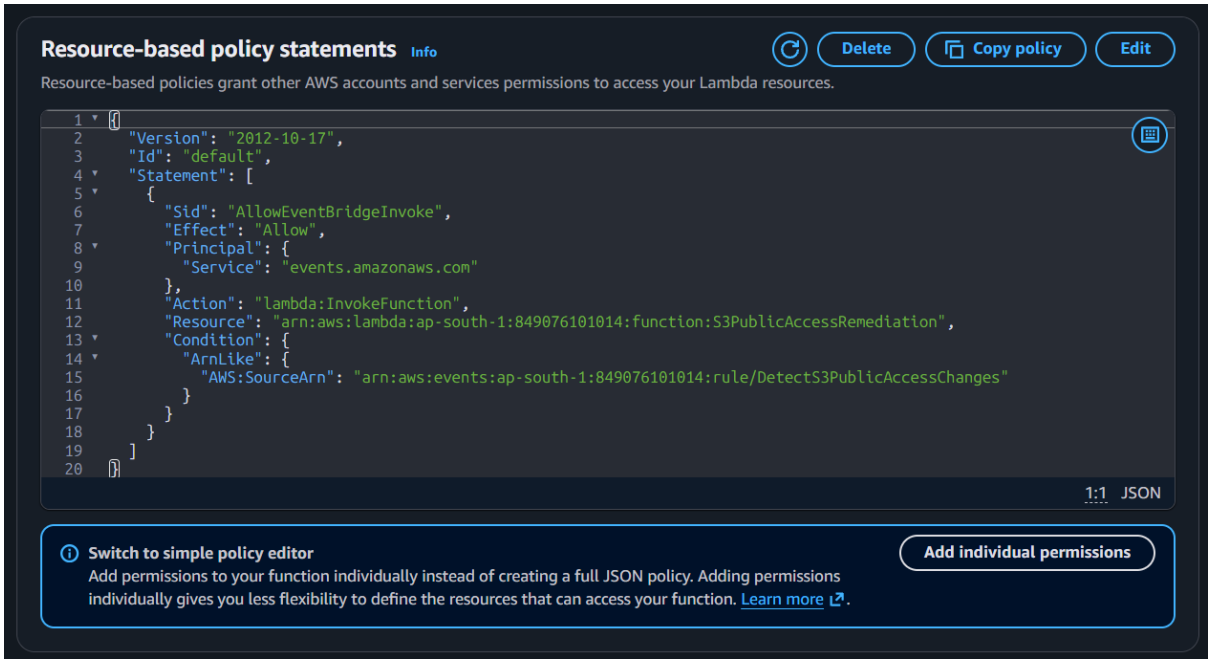


Figure 8: Lambda resource-based policy

5.5 CloudTrail trail

A trail named `s3-misconfig-trail` was created in ap-south-1 as a multi-region trail. It logs write management events only, so that read calls do not generate unnecessary events. Data events and Insights are disabled, and logs are delivered to a dedicated S3 log bucket created by the console, separate from the monitored bucket. The role this trail played in making real events reach the rule is discussed in Section 8.2.

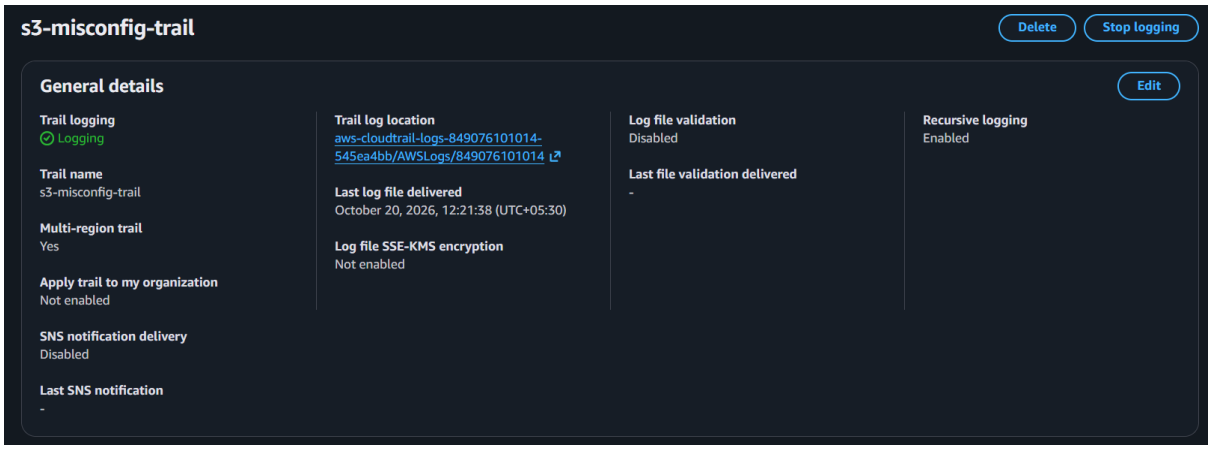


Figure 9: CloudTrail trail configuration

5.6 CloudWatch Logs

Two log groups hold the evidence (Table 7). Lambda creates its own group on first invocation, and EventBridge writes to the debug group.

Table 7: Log groups used

Log group	Content
/aws/lambda/S3PublicAccessRemediation	Received event, current settings, remediation result with verified flag, errors
/aws/events/s3-rule-debug	The raw JSON event delivered to the rule target

6. Security analysis

6.1 Security aspects addressed

The prototype addresses the aspects in Table 8. Each has a matching piece of evidence that can be shown in the demonstration.

Table 8: Security aspects covered by the demonstration

Aspect	How it is addressed	Evidence
Detective control	CloudTrail and an EventBridge rule notice the configuration change	Matched event in the debug log group
Corrective control	Lambda re-enables all four protections	S3 console back to On
Verification	Configuration is re-read and checked after the fix	Log line with verified true
Least privilege	Two S3 actions on one bucket; no wildcards	IAM policy JSON
Auditability	Both the change and the fix appear in CloudTrail; results are in CloudWatch	Two PutBucketPublicAccessBlock events
Exposure prevention	Account-level protection on; no public policy or ACL ever added	S3 and account settings

6.2 Least privilege and limited impact

If the function contained a defect, the damage it could do is limited by its role. It can read and write the Block Public Access configuration of one bucket and write logs. It cannot read objects, change bucket policies, touch other buckets or modify IAM. Because the remediation direction is always towards more protection, the worst realistic outcome of a bug is a bucket that stays as it was.

6.3 Threat considerations

Table 9: Threat considerations

Threat	How the design responds	Residual risk
A user weakens Block Public Access	Detected and reversed automatically	Short window before the fix, which depends on event delivery time
A public policy is added after BlockPublicPolicy is disabled	Remediation re-enables the setting, and no public policy was ever added in testing	Policy attached inside the window is not detected by this rule
The rule, trail or function is disabled or deleted	Those changes are themselves recorded by CloudTrail	No alert is raised by this prototype
Remediation loop	The function exits when the bucket is already secure	None observed

Threat	How the design responds	Residual risk
Over-permissive automation	Narrow IAM policy and bucket-scoped code	Role could still alter this bucket's setting

6.4 Further aspects the same design can cover

- Detect public bucket policies and ACLs, not only the Block Public Access flags.
- Enforce default encryption (SSE-S3 or KMS) and versioning on buckets.
- Send alerts through Amazon SNS so a security owner is told what changed.
- Watch other risky changes, such as IAM policy edits or security groups open to 0.0.0.0/0.
- Apply the rule to many buckets using tags instead of a fixed bucket name.
- Add AWS Config, AWS Security Hub and Amazon GuardDuty for continuous compliance and threat detection.

7. Testing and demonstration

7.1 Test strategy

Testing proceeded from the function alone to the complete chain, so that a failure could be located. Table 10 summarises the tests.

Table 10: Test cases

No.	Test	Method	Outcome
T1	Function in isolation	Invoke Lambda with a simulated EventBridge event	Passed: bucket read and written, result verified, log written
T2	Function on a secure bucket	Invoke Lambda while all four settings were on	Passed: response "Already secure", no change
T3	Event bus, rule and log target	Send a synthetic event to the default bus with a catch-all rule	Passed: event appeared in the log group
T4	Live chain, run 1	Disable Block public policy in the console and wait	Passed: setting restored without manual action
T5	Live chain, run 2	Repeat T4 to rule out a one-off result	Passed: setting restored without manual action

7.2 Demonstration procedure

1. Show that all four Block Public Access settings of the bucket are On.
2. Show the EventBridge rule, the Lambda function, its role and the CloudTrail trail.
3. In the S3 console, untick only Block public policy and save. Note the time.
4. Wait for event delivery. The recording can be edited to skip the wait, with a note on screen.
5. Refresh the S3 page: all four settings are On again.
6. Open the CloudWatch log streams and show the event, the settings found and the verified result.
7. Open CloudTrail Event history and show the change followed by the fix made by the Lambda role.

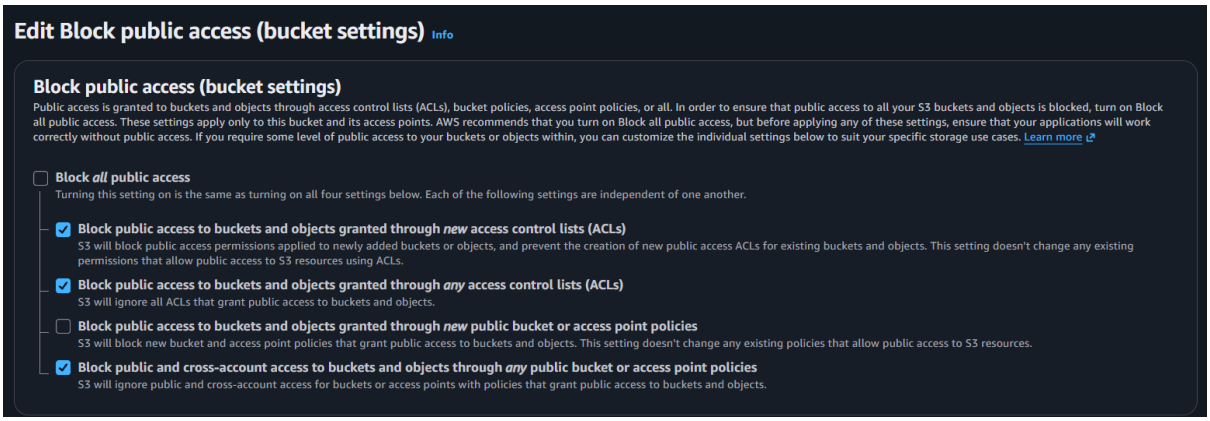


Figure 10: Misconfiguration step

7.3 Safety measures during testing

- Only Block public policy was disabled, and only for the time needed for each run.
- Account-level Block Public Access remained enabled.
- No public bucket policy or ACL was added, so no data was exposed.
- The bucket was returned to the fully secure state after every test.

8. Results and discussion

8.1 Observed results

In both live runs the disabled setting returned to On without any manual action. The following evidence shows the sequence. First, CloudTrail recorded the change.



Figure 11: CloudTrail event record of the misconfiguration

EventBridge matched it, and the debug log group received the raw event.

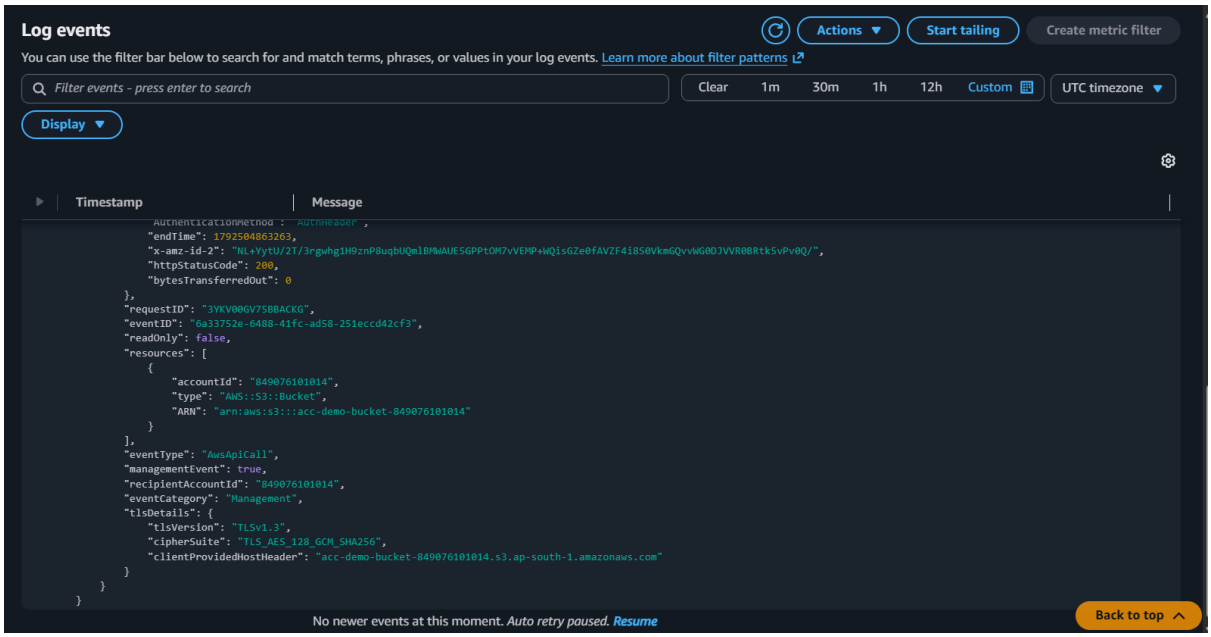


Figure 12: Matched event in the rule debug log group

The Lambda function then ran and logged the remediation result.

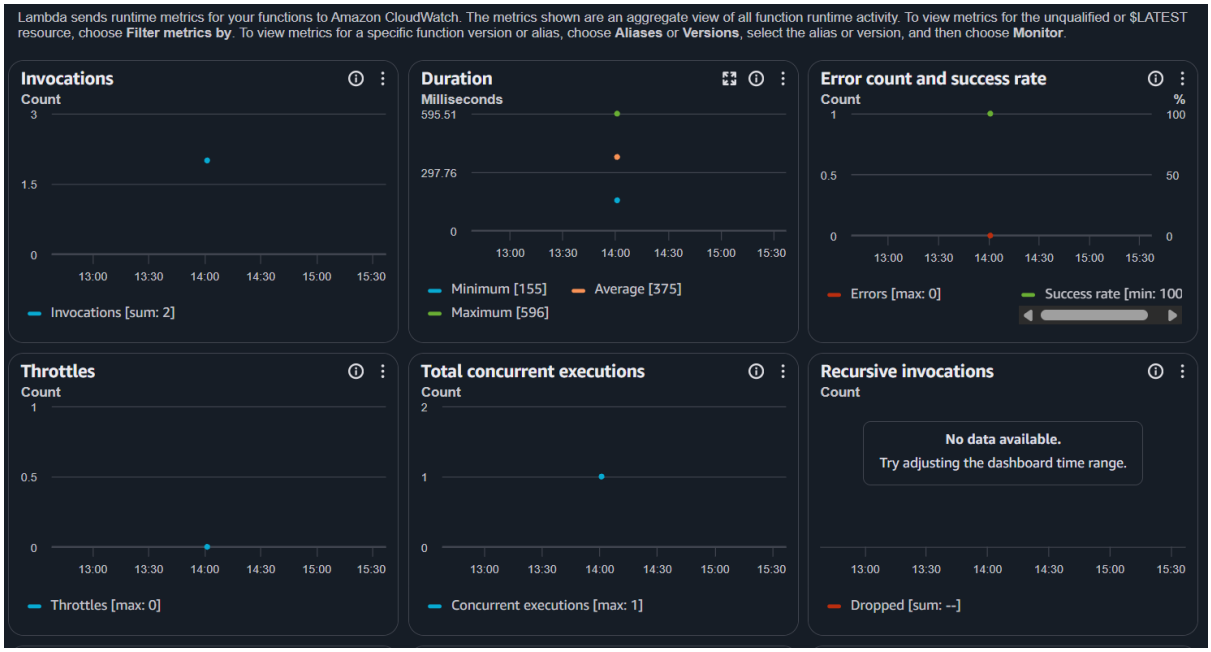


Figure 13: Lambda log output

The bucket returned to its secure state.

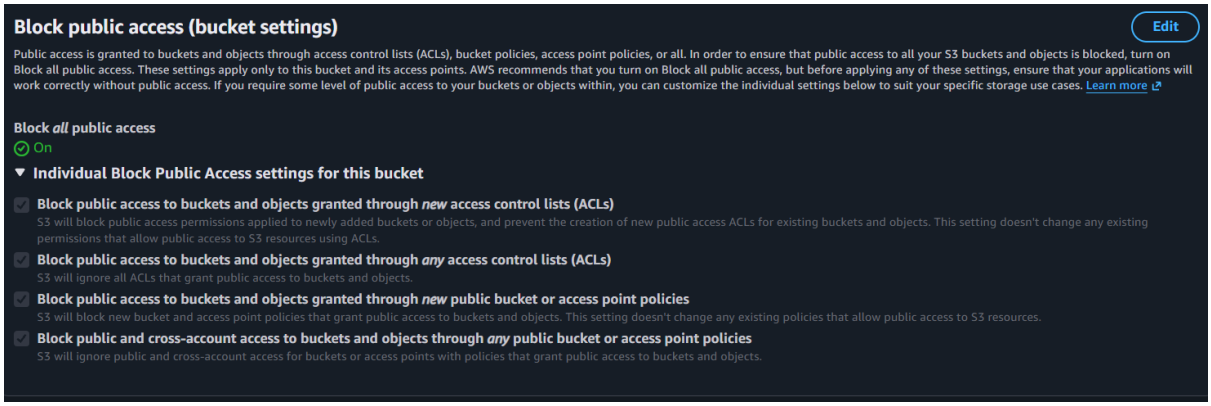


Figure 14: Bucket restored to the secure state

CloudTrail shows two events, the original change and the fix made by the function's role.

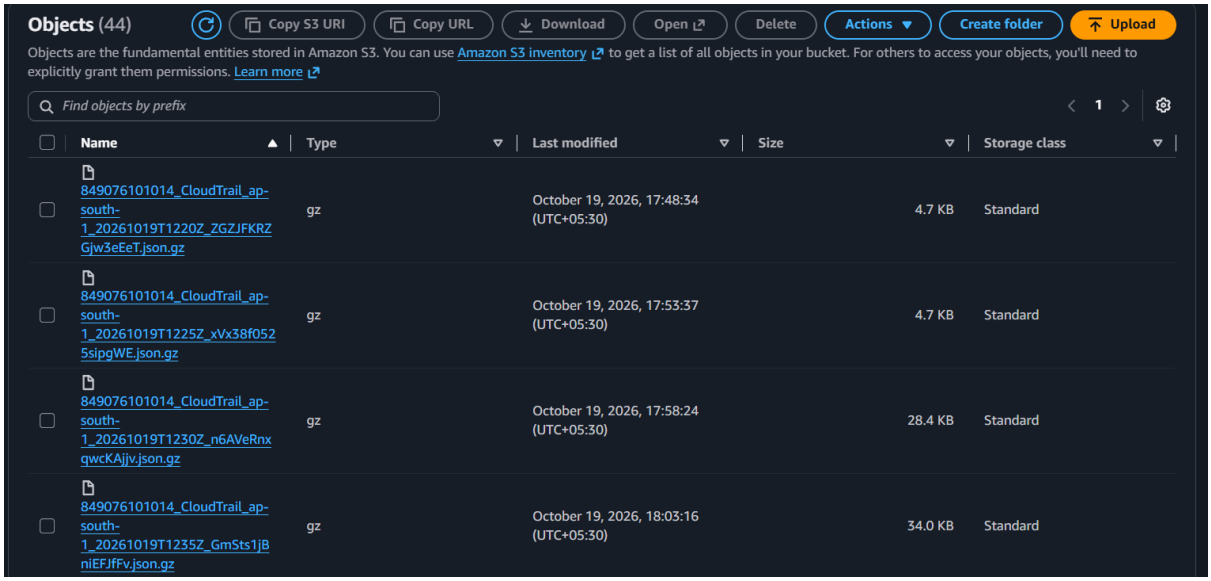


Figure 15: CloudTrail: change followed by remediation

Finally, the rule's metrics confirm that the match and invocation were counted.

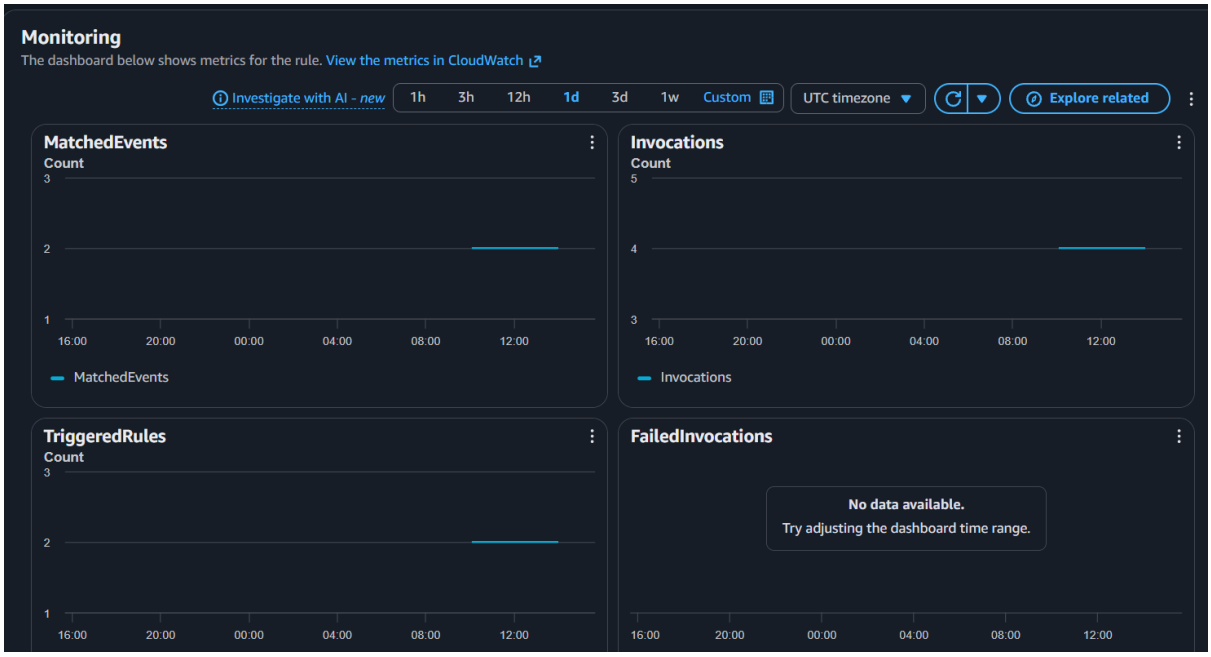


Figure 16: EventBridge rule monitoring

Measured values from the runs are recorded in Table 11. They are to be filled in from the logs and screenshots.

Table 11: Measured values

Measurement	Run 1	Run 2
Time from change to restored setting (observed)	[__ min]	[__ min]
Lambda duration (REPORT line in the log)	[__ ms]	[__ ms]
MatchedEvents / Invocations	[__ / __]	[__ / __]
Follow-up invocation logged "already secure"	[Yes / No]	[Yes / No]

The Lambda function itself takes a short time, because it makes only a few API calls. The delay seen between the change and the restoration is dominated by the time CloudTrail takes to deliver events, not by the remediation logic.

8.2 Troubleshooting record

Automatic invocation did not work at first, and the investigation is worth recording because it shows how the chain was isolated. Table 12 lists the observations and what each showed.

Table 12: Troubleshooting observations

Observation	Evidence	Finding	Action
No automatic invocation	Rule metrics showed no data; CloudTrail Event history did show the changes	The rule had never matched a live event	Checked rule, target and permissions in turn
Lambda trigger showed an error	Trigger said the rule could not be found	The rule had not been saved	Removed the stale trigger and created the rule again
Empty resource-based policy	Permissions tab said the policy was empty	EventBridge had no permission to invoke the function	Restored the events.amazonaws.com statement [state whether automatic or manual]
Empty log groups after a live change	Debug and catch-all log groups had no streams	Live events were not reaching the rules	Tested the bus with a synthetic event
Synthetic event succeeded	Event appeared in the catch-all log group	Bus, rule and log target all worked	Focused on delivery of real S3 events
CloudTrail trail created	After the trail was created, two live runs restored the setting automatically	Real events reached the rule only once the trail existed in our account	Kept the trail for the demonstration
CloudShell unavailable	Message that account verification was in progress	A new-account restriction	Used the S3 console to generate events

8.3 Discussion

The synthetic-event test separated two questions. The bus, the rule mechanism and the log target worked, so the open question was whether real S3 events reached them. In our account

they began to reach the rule after a CloudTrail trail with management events was created. We observed this order of events, and the trail is the change that preceded success in two consecutive runs. We did not run a controlled comparison to prove that the trail alone was responsible, and the account was still going through new-account verification during the work, which may also have affected service behaviour. For anyone reproducing the project, creating the trail early removes this uncertainty.

8.4 Limitations

Table 13: Limitations of the prototype

Limitation	Explanation
Single bucket	The bucket name is written into the function and the rule
Narrow detection	Only Block Public Access changes are handled; policies, ACLs, encryption and versioning are not
Not instantaneous	Restoration follows CloudTrail event delivery, which can take minutes
No notification	Nobody is alerted when a fix happens; the evidence is in the logs
Protected resources only	The trail, rule and function are not themselves protected from deletion by this design

9. Why cloud security is necessary

Cloud security matters because the speed and scale that make the cloud useful also make mistakes spread quickly. Five points explain why a control such as this one is worth building.

- **Shared responsibility.** The provider secures the platform, and the customer is responsible for configuration, identity and data [1]. A misconfigured bucket is the customer's exposure.
- **Speed.** Resources are created and changed in seconds, so human review cannot keep up with every change.
- **Misconfiguration risk.** Configuration mistakes are widely cited as a leading cause of cloud data exposure, and they need no attacker to be effective.
- **Consequences.** Exposed data can mean loss of privacy, legal obligations under data protection laws such as India's Digital Personal Data Protection Act, 2023, and loss of trust.
- **Automation.** Treating security checks as code, which detect, respond and record without waiting for a person, shortens the exposure window and leaves an audit trail.

10. Conclusion and future work

10.1 Conclusion

The project built a serverless detect, remediate, verify and log pipeline for S3 Block Public Access using CloudTrail, EventBridge, Lambda, IAM and CloudWatch Logs. The function was tested in isolation, the event path was tested with a synthetic event, and two live runs showed a weakened setting being restored without manual action, with the result verified and logged. The IAM role was limited to two actions on one bucket, and account-level protections stayed on throughout.

The troubleshooting record is part of the result. It showed that a rule, a trigger and a permission can each look correct while the chain is broken, and that separating the bus from the event source is an effective way to find the fault.

10.2 Future work

- Support many buckets and accounts by matching on tags and using AWS Organizations.
- Add Amazon SNS notifications that name the bucket, the user who made the change and the result.
- Extend detection to bucket policies, ACLs, encryption and versioning.
- Use AWS Config rules and AWS Security Hub for continuous compliance reporting.
- Define the whole stack as code, with CloudFormation or Terraform, so it can be deployed repeatably.
- Protect the trail, rule and function with service control policies or alarms on changes to them.

References

[1] Amazon Web Services, "Shared Responsibility Model." https://aws.amazon.com/compliance/shared-responsibility-model/

[2] Amazon Web Services, "Blocking public access to your Amazon S3 storage," Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

[3] Amazon Web Services, AWS CloudTrail User Guide. https://docs.aws.amazon.com/awscloudtrail/latest/userguide/

[4] Amazon Web Services, Amazon EventBridge User Guide. https://docs.aws.amazon.com/eventbridge/latest/userguide/

[5] Amazon Web Services, AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/

[6] Amazon Web Services, "Security best practices in IAM," IAM User Guide. https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

[7] Amazon Web Services, Boto3 documentation. https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

Appendix A: Lambda function source

The deployed function is reproduced below. Line wrapping has been adjusted for print; the logic is unchanged.

```
import json
import boto3
from botocore.exceptions import ClientError

s3 = boto3.client("s3")

BUCKET_NAME = "acc-demo-bucket-849076101014"

SECURE_SETTINGS = {
    "BlockPublicAcls": True,
    "IgnorePublicAcls": True,
    "BlockPublicPolicy": True,
    "RestrictPublicBuckets": True,
}


def get_settings():
    try:
        response = s3.get_public_access_block(Bucket=BUCKET_NAME)
        return response["PublicAccessBlockConfiguration"]
    except ClientError as e:
        code = e.response["Error"]["Code"]
        if code == "NoSuchPublicAccessBlockConfiguration":
            return {}
        raise


def is_secure(settings):
    return all(settings.get(k) is True for k in SECURE_SETTINGS)


def lambda_handler(event, context):
    print("Received event:", json.dumps(event))

    detail = event.get("detail", {}) or {}
    params = detail.get("requestParameters", {}) or {}
    bucket = params.get("bucketName")

    if bucket != BUCKET_NAME:
        print(f"Ignoring unrelated bucket: {bucket}")
        return {"statusCode": 200, "body": "Ignored"}

    current = get_settings()
    print("Current settings:", json.dumps(current))

    if is_secure(current):
        print("Bucket already secure. No remediation required.")
        return {"statusCode": 200, "body": "Already secure"}

    s3.put_public_access_block(
        Bucket=BUCKET_NAME,
        PublicAccessBlockConfiguration=SECURE_SETTINGS,
    )

    verified_settings = get_settings()
    verified = is_secure(verified_settings)

    result = {
```

```
        "bucket": BUCKET_NAME,
        "action": "restore_bucket_public_access_block",
        "triggered_by": detail.get("eventName"),
        "verified": verified,
        "settings": verified_settings,
    }
    print("Remediation result:", json.dumps(result))

    if not verified:
        raise RuntimeError("Remediation verification failed")

    return {"statusCode": 200, "body": json.dumps(result)}
```